Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Мухамедьянова А.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 06.11.26

Москва, 2026

Постановка задачи

Вариант 16.

Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса, иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в стандартный поток вывода.

В ЛР №3 вместо каналов используется общая память (shared memory). Для синхронизации используются семафоры.

Вариант 16) Правило проверки: строка должна оканчиваться на «.» или «;».

Общий метод и алгоритм решения

Использованные функции для организации межпроцессного взаимодействия через общую память:

- int shm_open(const char *name, int oflag, mode_t mode); - создаёт или открывает именованный объект разделяемой памяти и возвращает его файловый дескриптор.
- int ftruncate(int fd, off_t length); - устанавливает размер объекта разделяемой памяти. В программе используется для выделения 8192 байт.
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); - отображает объект разделяемой памяти в виртуальное адресное пространство процесса, обеспечивая доступ к его содержимому через указатель.
- void *memcpy(void *dest, const void *src, size_t n); - копирует заданное количество байтов из одной области памяти в другую. Используется для передачи строк и сообщений об ошибках через общую память.
- sem_t *sem_open(const char *name, int oflag, mode_t mode, int value); - создаёт или открывает именованный семафор для синхронизации процессов. В программе используются два семафора с начальным значением 0.

- int sem_wait(sem_t *sem); - уменьшает значение семафора на единицу, блокируя выполнение вызывающего процесса, если значение равно нулю.
- int sem_post(sem_t *sem); - увеличивает значение семафора на единицу, позволяя продолжить выполнение ожидающему процессу.
- int sem_close(sem_t *sem); - закрывает именованный семафор в текущем процессе.
- int sem_unlink(const char *name); - удаляет имя семафора из системы.
- int munmap(void *addr, size_t length); - удаляет отображение области разделяемой памяти из виртуального адресного пространства процесса.
- int shm_unlink(const char *name); - удаляет имя объекта разделяемой памяти из системы.

Программа состоит из двух независимых исполняемых файлов: родительского (parentshm.c) и дочернего (childshm.c). Взаимодействие между ними организовано через один именованный объект общей памяти (shared memory) и два именованных семафора:

- Общая память (8192 байта): разделена на две области по 4096 байт. Первая предназначена для передачи входных строк от родительского процесса дочернему, вторая — для передачи сообщений об ошибках в обратном направлении. В начале каждой области размещается 4-байтовое поле длины сообщения.
- sem_obr: семафор, сигнализирующий дочернему процессу о готовности входных данных к обработке.
- sem_zap: семафор, сигнализирующий родительскому процессу о завершении обработки данных.

1) Родительский процесс (parentshm.c)

- Формируются уникальные имена объекта общей памяти и двух семафоров с помощью идентификатора родительского процесса (PID).
- Считывается имя выходного файла, после чего он открывается для записи. При этом используются флаги, обозначающие: создание при отсутствии, очистка при существовании, добавление данных в конец файла).
- Вызовом shm_open() создаётся именованный объект общей памяти.
- Посредством ftruncate() размер объекта устанавливается равным 8192 байтам.
- Функция mmap() отображает объект общей памяти в виртуальное адресное пространство родительского процесса с разрешениями на чтение и запись.
- Через sem_open() создаются два именованных семафора sem_obr и sem_zap с начальными значениями 0.
- С помощью fork() порождается дочерний процесс.

2) Настройка дочернего процесса

- В ветке дочернего процесса (pid == 0) закрывается стандартный ввод stdin.
- Дескриптор открытого файла file_fd перенаправляется на стандартный вывод (1), так, чтобы корректные строки направлялись в файл.
- Исходный файловый дескриптор file_fd закрывается вызовом close().
- Формируется массив аргументов, содержащий путь к дочерней программе и имена объекта общей памяти и двух семафоров.
- Вызовом execv() запускается исполняемый файл ./childshm.

3) Обработка данных в дочерней программе (childshm.c)

- Дочерняя программа получает имена общей памяти и семафоров через аргументы командной строки argv.
- Вызовом shm_open() открывается созданный родителем объект разделяемой памяти.
- Функция mmap() отображает этот объект в виртуальное адресное пространство дочернего процесса.
- Через sem_open() открываются два существующих именованных семафора.
- Дочерний процесс ожидает поступления данных с помощью sem_wait(sem_obr).
- После получения сигнала из первой области общей памяти считываются длина сообщения и его содержимое. Данные копируются в локальный буфер функцией memcpy().
- Полученная строка проверяется на соответствие условию.
- При успешной проверке строка записывается в файл.
- При несоответствии правилу сообщение "Error: string must end with '.', or ';'\n" копируется во вторую область общей памяти, а в поле error_length записывается его длина.
- После завершения обработки дочерний процесс выполняет sem_post(sem_zap), уведомляя родителя о завершении обработки.

4) Передача данных, завершение и вывод ошибок

- Родительский процесс считывает строки пользователя из стандартного ввода и записывает их в первую область общей памяти функцией memcpy(), заполняя поле length количеством считанных байтов.
- После записи данных родитель вызывает sem_post(sem_obr), разрешая дочернему процессу приступить к обработке.
- Родитель блокируется на sem_wait(sem_zap) до получения сигнала о завершении обработки.

- После разблокировки родитель проверяет значение error_length во второй области общей памяти. Если оно больше нуля, сообщение об ошибке выводится в терминал через стандартный вывод ошибок и в выходной файл.
- Затем родитель переходит к чтению следующей строки и повторяет обмен.
- При достижении конца ввода (EOF) родитель записывает в поле length специальное значение UINT32_MAX и уведомляет дочерний процесс через sem_post(sem_obr).
- Дочерний процесс распознаёт сигнал завершения, выходит из цикла обработки и выполняет sem_post(sem_zap), разблокируя родителя.
- Дочерняя программа закрывает семафоры через sem_close(), удаляет отображение памяти через munmap() и закрывает дескриптор общей памяти.
- Родитель дожидается завершения дочернего процесса с помощью wait(), после чего закрывает семафоры, удаляет их имена через sem_unlink(), освобождает отображение общей памяти и удаляет её именованный объект через shm_unlink().

Код программы

parentshm.c

```
#include <fcntl.h>
#include <stdint.h>
#include <stdbool.h>

#include <stdlib.h>
#include <string.h>
#include <errno.h>

#include <unistd.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <sys/mman.h>

#define SHM_SIZE 8192
#define PART_SIZE 4096
#define BUFFER_SIZE 4092

int main(){

   char shm_name[64] = "/shm-";
   char sem_zap_name[64] = "/sem_zap-";
   char sem_obr_name[64] = "/sem_obr-";
   pid_t ppid = getpid();

   int i_shm = 5;
   int i_sem_zap = 9;
   int i_sem_obr = 9;
   do{
      int didgit = ppid%10;
      shm_name[i_shm++] = '0' + didgit;
      sem_zap_name[i_sem_zap++] = '0' + didgit;
      sem_obr_name[i_sem_obr++] = '0' + didgit;
      ppid /= 10;
   }while (ppid > 0);

   shm_name[i_shm] = '\0';
   sem_zap_name[i_sem_zap] = '\0';
   sem_obr_name[i_sem_obr] = '\0';

   char buffer[BUFFER_SIZE];
   int filename_len = read(0, buffer, BUFFER_SIZE -1);
   if (filename_len <= 0){
      write(2, "filename read error\n", 20);
      exit(1);
   }
   if (buffer[filename_len-1] == '\n'){
```

```
    buffer[filename_len-1] = '\0';
}
else {
    buffer[filename_len] = '\0';
}

int file_fd = open(buffer, O_WRONLY | O_CREAT | O_TRUNC | O_APPEND, 0600);
if (file_fd == -1){
    write(2, "file open error\n", 16);
    exit(1);
}

int shm = shm_open(shm_name, O_RDWR | O_CREAT | O_EXCL, 0600);
if(shm == -1){
    write(2, "SHM open/create error\n", 22);
    close(file_fd);
    exit(1);
}

if (ftruncate(shm, SHM_SIZE) == -1){
    write(2, "SHM resize error\n", 17);
    close(shm);
    shm_unlink(shm_name);
    close(file_fd);
    exit(1);
}

char *shm_buf = mmap(NULL, SHM_SIZE, PROT_WRITE | PROT_READ, MAP_SHARED, shm, 0);
if (shm_buf == MAP_FAILED){
    write(2, "SHM map error\n", 14);
    close(shm);
    shm_unlink(shm_name);
    close(file_fd);
    exit(1);
}

sem_t *sem_zap = sem_open(sem_zap_name, O_CREAT | O_EXCL, 0600, 0);
if (sem_zap == SEM_FAILED){
    write(2, "SEM_zap open error\n", 15);
    munmap(shm_buf, SHM_SIZE);
    close(shm);
    shm_unlink(shm_name);
    close(file_fd);
    exit(1);
}

sem_t *sem_obr = sem_open(sem_obr_name, O_CREAT | O_EXCL, 0600, 0);
if (sem_obr == SEM_FAILED){
    write(2, "SEM_obr open error\n", 15);
    sem_close(sem_zap);
```

```
    sem_unlink(sem_zap_name);
    munmap(shm_buf, SHM_SIZE);
    close(shm);
    shm_unlink(shm_name);
    close(file_fd);
    exit(1);
}

pid_t pid = fork();
if (pid == -1){
    write(2, "fork error\n", 11);
    close(file_fd);
    munmap(shm_buf, 8192);
    close(shm);
    shm_unlink(shm_name);
    sem_close(sem_zap);
    sem_close(sem_obr);
    sem_unlink(sem_zap_name);
    sem_unlink(sem_obr_name);
    exit(1);
}

if (pid == 0){
    close(STDIN_FILENO);
    if(dup2(file_fd, 1) == -1){
        write(2, "dup2 stdout error\n", 18);
        exit(1);
    }
    close(file_fd);

    char *args[] = {
        "./childshm",
        shm_name,
        sem_zap_name,
        sem_obr_name,
        NULL
    };
    execv("./childshm", args);
    write(2, "exec error\n", 11);
    _exit(1);
}

if(dup2(file_fd, 1) == -1){
        write(2, "dup2 stdout error\n", 18);
        exit(1);
    }
close(file_fd);

bool running = true;
while(running){
```

```
        char buf[PART_SIZE - sizeof(uint32_t)];
        ssize_t bytes = read(0, buf, sizeof(buf));
        if (bytes == -1){
            write(2, "stdin read error\n", 17);
            exit(1);
        }
        uint32_t *length = (uint32_t *)shm_buf;
        char *str = shm_buf + sizeof(uint32_t);

        uint32_t *error_length = (uint32_t *)(shm_buf + PART_SIZE);
        char *err = shm_buf + PART_SIZE + sizeof(uint32_t);
        if(bytes > 0){
            *length = bytes;
            memcpy(str, buf, bytes);
        } else {
            *length = UINT32_MAX;
            running = false;
        }
        sem_post(sem_obr);
        sem_wait(sem_zap);

        if (*error_length > 0) {
        write(2, err, *error_length);
        write(1, err, *error_length);
}
    }

    wait(NULL);
    sem_unlink(sem_zap_name);
    sem_unlink(sem_obr_name);
    sem_close(sem_zap);
    sem_close(sem_obr);
    munmap(shm_buf, SHM_SIZE);
    shm_unlink(shm_name);
    close(shm);


    return 0;
}
```

childshm.c

```
#include <stdint.h>
#include <stdbool.h>

#include <stdlib.h>
#include <string.h>

#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
```

```c
#include <semaphore.h>
#include <sys/mman.h>

#define SHM_SIZE 8192
#define PART_SIZE 4096
#define BUFFER_SIZE 4092

int main(int argc, char *argv[]){
    if (argc != 4){
        return 1;
    }

    const char *shm_name = argv[1];
    const char *sem_zap_name = argv[2];
    const char *sem_obr_name = argv[3];

    int shm = shm_open(shm_name, O_RDWR, 0600);
    if(shm == -1){
        write(2, "SHM open error\n", 15);
        exit(1);
    }

    char *shm_buf = mmap(NULL, SHM_SIZE, PROT_WRITE | PROT_READ, MAP_SHARED, shm, 0);
    if (shm_buf == MAP_FAILED){
        write(2, "SHM map error\n", 14);
        close(shm);
        exit(1);
    }

    sem_t *sem_zap = sem_open(sem_zap_name, 0);
    if (sem_zap == SEM_FAILED){
        write(2, "SEM_zap open error\n", 15);
        munmap(shm_buf, SHM_SIZE);
        close(shm);
        exit(1);
    }

    sem_t *sem_obr = sem_open(sem_obr_name, 0);
    if (sem_obr == SEM_FAILED){
        write(2, "SEM_obr open error\n", 15);
        sem_close(sem_zap);
        munmap(shm_buf, SHM_SIZE);
        close(shm);
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    bool running = true;
    while(running){
```

```
        sem_wait(sem_obr);
        uint32_t *length = (uint32_t *)shm_buf;
        char *str = shm_buf + sizeof(uint32_t);
        char *err = shm_buf + PART_SIZE + sizeof(uint32_t);
        uint32_t *error_length = (uint32_t *)(shm_buf + PART_SIZE);
        err[0] = '\0';
        *error_length = 0;

        if(*length == UINT32_MAX){
            running = false;
        } else if (*length > 0){
            memcpy(buffer, str, *length);
            int start = 0;
            int end = start;
            while(end < *length && buffer[end] != '\n'){
                end++;
            }
            int last = end-1;
            if(last >= start && (buffer[last] == '.' || buffer[last] == ';')){
                int res = write(1, buffer, *length);
                if (res < 0){
                    write(2, "file write error\n", 17);
                    exit(1);
                }
            } else{
                const char errm[] = "Error: string must end with '.', or ';'\n";
                memcpy(err, errm, sizeof(errm) - 1);
                *error_length = sizeof(errm) - 1;
            }
        }
        sem_post(sem_zap);

    }
    sem_close(sem_zap);
    sem_close(sem_obr);
    munmap(shm_buf, SHM_SIZE);
    close(shm);
    return 0;
}
```

Протокол работы программы

Тестирование:

Script started on 2026-11-06 23:21:59+03:00 [TERM="xterm-256color" TTY="/dev/pts/4" COLUMNS="150" LINES="21"]
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

allie@parsley:~/ос_код/лаба3$ ./parentshm
result.txt
a;
b.
c
Error: string must end with '.', or ';'
c;
c.
;
.
!
Error: string must end with '.', or ';'
allie@parsley:~/ос_код/лаба3$ exit
exit

Script done on 2026-11-06 23:22:59+03:00 [COMMAND_EXIT_CODE="0"]

Вывод

В ходе лабораторной работы была успешно реализована программа на языке C для межпроцессного взаимодействия с использованием общей памяти (shared memory), файлового отображения (file mapping) и синхронизации в роли двух именованных семафор. Родительский процесс считывает данные пользователя, передает их в общею память; дочерний процесс считывает данные из памяти, обрабатывает их и передает в ту же память сообщения об ошибках. Дочерний процесс проверяет строки на соответствие заданному правилу (окончание на «.» или «;»), сохраняя валидные строки в файл. Для выполнения лабораторной работы были изучены и использованы функции: int shm_open(); int ftruncate(); void *mmap(); void *memcpy(); sem_t *sem_open(); int sem_wait(); int sem_post(); int

sem_close(); int sem_unlink(); int munmap(); int shm_unlink(). Программа работает еорректно, поставленная задача выполнена.